
TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPIUFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC

FICHA PRÁTICA 3

MODEL AND ENTITY FRAMEWORK

A ideia é mostrar a sequência completa:

1. **criar as classes do domínio**
2. **configurar o Entity Framework Core**
3. **gerar a base de dados**
4. **implementar a aplicação MVC**

Vou usar um exemplo mínimo com estas entidades:

- **Categoria**
- **Produto**

1. Criar as classes do domínio

As **classes do domínio** representam os objetos principais do problema real. Num projeto MVC, estas classes costumam ficar na pasta **Models**.

Classe Categoria

```
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace LojaOnline.Models
{
    public class Categoria
    {
        public int Id { get; set; }

        [Required]
        [StringLength(100)]
        public string Nome { get; set; } = string.Empty;

        // Relação: uma categoria pode ter muitos produtos
        public List<Produto> Produtos { get; set; } = new List<Produto>();
    }
}
```

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

Classe Produto

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace LojaOnline.Models
{
    public class Produto
    {
        public int Id { get; set; }

        [Required]
        [StringLength(150)]
        public string Nome { get; set; } = string.Empty;

        [Column(TypeName = "decimal(10,2)")]
        [Range(0.01, 999999.99)]
        public decimal Preco { get; set; }

        [Range(0, 9999)]
        public int Stock { get; set; }

        // Chave estrangeira
        [Display(Name = "Categoria")]
        public int CategoriaId { get; set; }

        // Propriedade de navegação
        public Categoria? Categoria { get; set; }
    }
}
```

Interpretação pedagógica

O que compreender:

- cada classe representa uma **entidade do domínio**
- cada propriedade simples representa um **atributo**
- CategoriaId é uma **chave estrangeira**
- Categoria e Produtos são **propriedades de navegação**
- a relação é **1:N** (uma categoria pode ter muitos produtos)

2. Configurar o Entity Framework Core

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

O Entity Framework Core é a tecnologia que vai permitir:

- mapear as classes para tabelas
- criar a base de dados
- fazer operações CRUD

Criar a classe LojaContext

Esta classe deve herdar de `DbContext`.

Pasta sugerida: **Data**

```
using LojaOnline.Models;
using Microsoft.EntityFrameworkCore;

namespace LojaOnline.Data
{
    public class LojaContext : DbContext
    {
        public LojaContext(DbContextOptions<LojaContext> options)
            : base(options)
        {
        }

        public DbSet<Categoria> Categorias { get; set; }
        public DbSet<Produto> Produtos { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<Categoria>().ToTable("Categoria");
            modelBuilder.Entity<Produto>().ToTable("Produto");
        }
    }
}
```

Registar o contexto em Program.cs

Num projeto ASP.NET Core MVC moderno, a configuração é feita em `Program.cs`.

```
using LojaOnline.Data;
using Microsoft.EntityFrameworkCore;
```

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

```
var builder = WebApplication.CreateBuilder(args);

// Adicionar serviços MVC
builder.Services.AddControllersWithViews();

// Configurar Entity Framework Core com SQL Server
builder.Services.AddDbContext<LojaContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("LojaContext")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Adicionar a string de ligação em appsettings.json

```
{
  "ConnectionStrings": {
    "LojaContext":
"Server=(localdb)\\mssqllocaldb;Database=LojaOnlineDB;Trusted_Connection=True;MultipleActiveResultSets=true"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

3. Gerar a base de dados

Depois de criar as classes e configurar o contexto, é necessário gerar a base de dados.

Pacotes NuGet normalmente necessários

Num projeto MVC com SQL Server, deverá ser preciso:

- `Microsoft.EntityFrameworkCore.SqlServer`
- `Microsoft.EntityFrameworkCore.Tools`

Criar a migration

No **Package Manager Console** ou terminal:

```
Add-Migration InitialCreate
```

ou no terminal:

```
dotnet ef migrations add InitialCreate
```

Atualizar a base de dados

```
Update-Database
```

ou

```
dotnet ef database update
```

O que isto faz?

Add-Migration

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

Analisa as classes e o `DbContext` e cria um ficheiro de migration com instruções para criar as tabelas.

Update-Database

Aplica essas instruções à base de dados real.

Resultado esperado

A base de dados ficará com, por exemplo:

Tabela Categoria

- Id
- Nome

Tabela Produto

- Id
- Nome
- Preço
- Stock
- CategoriaId

com uma relação entre `Produto` e `Categoria`.

4. Implementar a aplicação MVC

Agora entra a parte do ASP.NET Core MVC.

Deverás criar:

- **Controller**
- **Views**
- operações CRUD

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

Exemplo simples para Produto.

4.1 Criar o ProdutosController

```
using LojaOnline.Data;
using LojaOnline.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;

namespace LojaOnline.Controllers
{
    public class ProdutosController : Controller
    {
        private readonly LojaContext _context;

        public ProdutosController(LojaContext context)
        {
            _context = context;
        }

        // LISTAR
        public async Task<IActionResult> Index()
        {
            var produtos = _context.Produtos.Include(p => p.Categoria);
            return View(await produtos.ToListAsync());
        }

        // DETALHES
        public async Task<IActionResult> Details(int? id)
        {
            if (id == null) return NotFound();

            var produto = await _context.Produtos
                .Include(p => p.Categoria)
                .FirstOrDefaultAsync(m => m.Id == id);

            if (produto == null) return NotFound();

            return View(produto);
        }

        // GET: Criar
        public IActionResult Create()
        {
            ViewData["CategoriaId"] = new SelectList(_context.Categorias, "Id",
```

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

```
"Nome");  
    return View();  
}  
  
// POST: Criar  
[HttpPost]  
[ValidateAntiForgeryToken]  
public async Task<IActionResult>  
Create([Bind("Id, Nome, Preco, Stock, CategoriaId")] Produto produto)  
{  
    if (ModelState.IsValid)  
    {  
        _context.Add(produto);  
        await _context.SaveChangesAsync();  
        return RedirectToAction(nameof(Index));  
    }  
  
    ViewData["CategoriaId"] = new SelectList(_context.Categorias, "Id",  
"Nome", produto.CategoriaId);  
    return View(produto);  
}  
  
// GET: Editar  
public async Task<IActionResult> Edit(int? id)  
{  
    if (id == null) return NotFound();  
  
    var produto = await _context.Produtos.FindAsync(id);  
    if (produto == null) return NotFound();  
  
    ViewData["CategoriaId"] = new SelectList(_context.Categorias, "Id",  
"Nome", produto.CategoriaId);  
    return View(produto);  
}  
  
// POST: Editar  
[HttpPost]  
[ValidateAntiForgeryToken]  
public async Task<IActionResult> Edit(int id,  
[Bind("Id, Nome, Preco, Stock, CategoriaId")] Produto produto)  
{  
    if (id != produto.Id) return NotFound();  
  
    if (ModelState.IsValid)  
    {  
        try  
        {  
            _context.Update(produto);  
            await _context.SaveChangesAsync();  
        }  
    }  
}
```

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

```
        catch (DbUpdateConcurrencyException)
        {
            if (!_context.Produtos.Any(e => e.Id == produto.Id))
                return NotFound();
            else
                throw;
        }

        return RedirectToAction(nameof(Index));
    }

    ViewData["CategoriaId"] = new SelectList(_context.Categorias, "Id",
"Nome", produto.CategoriaId);
    return View(produto);
}

// GET: Apagar
public async Task<IActionResult> Delete(int? id)
{
    if (id == null) return NotFound();

    var produto = await _context.Produtos
        .Include(p => p.Categoria)
        .FirstOrDefaultAsync(m => m.Id == id);

    if (produto == null) return NotFound();

    return View(produto);
}

// POST: Confirmar apagar
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var produto = await _context.Produtos.FindAsync(id);
    if (produto != null)
    {
        _context.Produtos.Remove(produto);
        await _context.SaveChangesAsync();
    }

    return RedirectToAction(nameof(Index));
}
}
```

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

4.2 Criar as Views

As views podem ser geradas automaticamente com **scaffolding**, ou feitas manualmente.

As principais views para CRUD são:

- Index.cshtml
- Details.cshtml
- Create.cshtml
- Edit.cshtml
- Delete.cshtml

Vou dar um exemplo simples da view Index.

Views/Produtos/Index.cshtml

```
@model IEnumerable<LojaOnline.Models.Produto>

@{
    ViewData["Title"] = "Lista de Produtos";
}

<h1>Lista de Produtos</h1>

<p>
    <a asp-action="Create" class="btn btn-primary">Novo Produto</a>
</p>

<table class="table">
    <thead>
        <tr>
            <th>Nome</th>
            <th>Preço</th>
            <th>Stock</th>
            <th>Categoria</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
@foreach (var item in Model)
{
    <tr>
        <td>@item.Nome</td>
```

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI

UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC

```

        <td>@item.Preco €</td>
        <td>@item.Stock</td>
        <td>@item.Categoria?.Nome</td>
        <td>
            <a asp-action="Edit" asp-route-id="@item.Id">Editar</a> |
            <a asp-action="Details" asp-route-id="@item.Id">Detalhes</a> |
            <a asp-action="Delete" asp-route-id="@item.Id">Apagar</a>
        </td>
    </tr>
}
</tbody>
</table>

```

4.3 Exemplo da view Create.cshtml

```

@model LojaOnline.Models.Produto

@{
    ViewData["Title"] = "Criar Produto";
}

<h1>Criar Produto</h1>

<form asp-action="Create">
    <div class="mb-3">
        <label asp-for="Nome" class="form-label"></label>
        <input asp-for="Nome" class="form-control" />
        <span asp-validation-for="Nome" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Preco" class="form-label"></label>
        <input asp-for="Preco" class="form-control" />
        <span asp-validation-for="Preco" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Stock" class="form-label"></label>
        <input asp-for="Stock" class="form-control" />
        <span asp-validation-for="Stock" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="CategoriaId" class="form-label"></label>
        <select asp-for="CategoriaId" class="form-control" asp-
items="ViewBag.CategoriaId"></select>
        <span asp-validation-for="CategoriaId" class="text-danger"></span>
    </div>

```

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

```
</div>

<button type="submit" class="btn btn-success">Guardar</button>
<a asp-action="Index" class="btn btn-secondary">Voltar</a>
</form>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

5. Fluxo completo do processo

Podem visualizar o processo assim:

Etapa 1 — Domínio

Criam as classes:

- Categoria
- Produto

Estas classes representam o problema real.

Etapa 2 — Persistência

Criam:

- LojaContext
- DbSet<Categoria>
- DbSet<Produto>

Isto permite ao Entity Framework Core saber quais são as tabelas da base de dados.

Etapa 3 — Base de dados

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

Executam:

- `Add-Migration InitialCreate`
- `Update-Database`

A base de dados é criada automaticamente.

Etapa 4 — Aplicação MVC

Criar:

- `ProdutosController`
- views para listar, criar, editar, apagar e ver detalhes

Aqui já existe interação completa entre:

- **Model** → classes `Produto` e `Categoria`
- **Controller** → `ProdutosController`
- **View** → páginas `.cshtml`

6. Relação com o padrão MVC

Exemplo para explicar o MVC.

Model

Representa os dados e a lógica estrutural:

- `Produto`
- `Categoria`
- `LojaContext`

View

Mostra os dados ao utilizador:

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

- `Index.cshtml`
- `Create.cshtml`
- `etc.`

Controller

Recebe os pedidos HTTP, processa-os e escolhe a resposta:

- `ProdutosController`

7. Sequência de trabalho resumida

Passo 1

Criar o projeto ASP.NET Core MVC

Passo 2

Criar a pasta `Models` e adicionar:

- `Categoria.cs`
- `Produto.cs`

Passo 3

Criar a pasta `Data` e adicionar:

- `LojaContext.cs`

Passo 4

Configurar:

- `Program.cs`
- `appsettings.json`

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

Passo 5

Instalar pacotes EF Core

Passo 6

Executar:

```
dotnet ef migrations add InitialCreate  
dotnet ef database update
```

Passo 7

Criar o controller e as views

Passo 8

Testar o CRUD no browser

8. Exemplo – Desenvolvimentos futuros

Depois deste exemplo base, podem expandir a loja online com:

- Cliente
- Encomenda
- LinhaEncomenda
- Carrinho
- Utilizador
- autenticação
- upload de imagem de produto
- pesquisa e filtros

Ou seja, o exemplo é pequeno, mas já contém a arquitetura correta.

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPIUFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC

9. Proposta de passos a seguir

1ª etapa:

- **Categoria**
- **Produto**

2ª etapa:

- **Cliente**
- **Encomenda**
- **LinhaEncomenda**

A entidade `Encomenda` com relação N:M resolvida por `LinhaEncomenda` já é mais exigente.